

TalentBridge — AI Integration Report

Date: 2026-07-14

This report documents the AI integration in TalentBridge's NestJS backend: the four Gemini-backed features, their prompt templates, context-building strategy, fallback/degradation behavior, the real prompt/model iteration history recovered from git, and the data-privacy implications of sending recruiting data to a third-party LLM API.

Source of truth for everything below: `backend/src/modules/ai/prompts.ts` , `ai.service.ts` , `ai.controller.ts` , `ai.types.ts` , `interview-question-fallbacks.ts` , `PROMPTS.md` , and `backend/test/ai.e2e-spec.ts` .

1. Full Prompt Templates

There are four AI features, each with one exported prompt-builder function in `backend/src/modules/ai/prompts.ts` . All four share two conventions (stated verbatim in the file's header comment): every prompt ends with an explicit "respond with JSON only, no markdown fences" instruction plus an exact JSON shape, and every prompt states output-size constraints as hard numbers (e.g. "exactly 9 questions") rather than soft language like "a few."

1.1 Job Description Generator — `buildJobDescriptionPrompt`

```
`You are a technical recruiter writing an inclusive, gender-neutral job description for an SME (small/medium business).
```

```
Role: ${input.jobTitle}
Department: ${input.department}
Seniority level: ${input.level}
Required skills: ${input.requiredSkills.join(', ')}
${input.cultureNotes ? `Company culture notes: ${input.cultureNotes}` : ''}
```

```
Write the job description using inclusive, gender-neutral language (use "you/they", never "he/she"; avoid gendered or ableist idioms like "rockstar" or "ninja"; avoid unnecessary degree requirements that could exclude non-traditional candidates).
```

```
Respond with JSON only, no markdown fences, in exactly this shape:
{
  "roleSummary": "2-3 sentence overview of the role and its impact",
  "keyResponsibilities": ["5 to 7 bullet strings"],
  "requiredQualifications": ["bullet strings – only what's truly required"],
  "preferredQualifications": ["bullet strings – nice-to-haves"],
  "whatWeOffer": ["bullet strings – compensation/culture/benefits framing"]
}`
```

1.2 CV Screening & Scorer — `buildCvScorePrompt`

```
`You are an unbiased technical recruiter screening a candidate's CV against a specific job. Judge only skills, experience, and demonstrated ability – never infer or comment on age, gender, ethnicity, nationality, or any protected characteristic, even if such details appear in the CV text.
```

```
Job title: ${input.jobTitle}  
Required skills: ${input.requiredSkills.join(', ')}  
Responsibilities: ${input.responsibilities}
```

```
Candidate CV (extracted text, formatting may be imperfect):
```

```
""  
${input.cvExtractedText}  
""
```

```
Extract the candidate's profile and score their fit for this specific job. matchScore is 0-100 based on how well their demonstrated skills/experience align with the required skills and responsibilities above – not a general CV quality score.
```

```
Respond with JSON only, no markdown fences, in exactly this shape:
```

```
{  
  "candidateName": "string, or \"Unknown\" if not found in the text",  
  "yearsOfExperience": <integer, best estimate from the CV>,  
  "topSkills": ["exactly 5 strings, the candidate's strongest relevant skills"],  
  "educationLevel": "string, e.g. \"Bachelor's in Computer Science\", or \"Not specified\"",  
  "lastRole": "string, most recent job title (and company if present)",  
  "matchScore": <integer 0-100>,  
  "topStrengths": ["exactly 3 strings – specific reasons this candidate fits"],  
  "topGaps": ["exactly 2 strings – specific gaps against the required skills/responsibilities"]  
}
```

1.3 Interview Question Suggester — buildInterviewQuestionsPrompt

```
// typeGuidance lookup, keyed by interviewType:  
const typeGuidance: Record<InterviewQuestionsPromptInput['interviewType'], string> = {  
  technical:  
    'Focus on hands-on technical depth against the required skills – problem-solving, system design, or coding-adjacent reasoning appropriate to the role.',  
  behavioural:  
    'Focus on past behaviour and soft skills – collaboration, conflict resolution, ownership, communication. Favor STAR-style prompts ("tell me about a time...").',  
  final:  
    'Focus on role fit, motivation, culture alignment, and closing-stage concerns (career goals, working style, what would make them decline an offer).',  
};
```

```
`You are a hiring manager preparing for a ${input.interviewType} interview.
```

```
Job title: ${input.jobTitle}
```

```
Required skills: ${input.requiredSkills.join(', ')}
Responsibilities: ${input.responsibilities}

Candidate summary (from their CV):
""
${input.cvSummary}
""

${typeGuidance[input.interviewType]}

Generate exactly 9 interview questions tailored to this specific candidate and role –
reference their actual background where it makes a question sharper, not generic questions
that could apply to anyone.

Respond with JSON only, no markdown fences, in exactly this shape:
{
  "questions": [
    { "question": "string", "listenFor": "1-2 sentence note on what a strong answer
demonstrates" }
  ]
}
```

1.4 Offer Letter Drafter — `buildOfferLetterPrompt`

```
`You are an HR specialist drafting a formal job offer letter.

Candidate: ${input.candidateName}
Role: ${input.roleTitle}
Company: ${input.companyName}
Annual salary: ${input.salary}
Start date: ${input.startDate}
Probation period: ${input.probationPeriod}
Benefits: ${input.benefits.join(', ')}

Write a complete, formal offer letter. This is a DRAFT for a recruiter to review before
sending – do not include any language implying the offer has already been sent or accepted.

Respond with JSON only, no markdown fences, in exactly this shape:
{
  "letterText": "the full offer letter as plain text, with sections for introduction, role
details, compensation, benefits, conditions of employment, and acceptance instructions,
using \\n for line breaks"
}
```

2. Context Management Strategy

TalentBridge does not maintain a conversational context window or vector store — each feature is a single, stateless request/response call to Gemini. "Context management" here means: what gets pulled from the database, how it is shaped into the prompt string, and what guardrails apply before interpolation.

2.1 Per-feature context building (in `AiController`)

Feature	Data source	Shaping before interpolation
Job Description Generator	None from the DB — <code>GenerateJobDescriptionDto</code> (jobTitle, department, requiredSkills, level, optional cultureNotes) is passed straight from the request body into <code>buildJobDescriptionPrompt</code> .	<code>requiredSkills</code> array is joined with <code>, `</code> into a flat string; <code>cultureNotes</code> is only interpolated at all if present (conditional template line), otherwise omitted entirely rather than sent as <code>"undefined"</code> .
CV Screening & Scorer	<code>ApplicationsService.findById(applicationId)</code> → the <code>Application</code> entity (for <code>cvExtractedText</code>), then <code>JobsService.findById(application.jobPostingId)</code> → the <code>Job</code> entity (for title, requiredSkills, responsibilities).	Guard clause: if <code>application.cvExtractedText</code> is falsy, the controller returns <code>{ scored: false, message: 'No CV text has been extracted...' }</code> immediately — no Gemini call, no prompt built. <code>requiredSkills</code> is joined with <code>, `</code> . The raw extracted CV text is passed through untouched (no truncation, no PII stripping) inside a <code>""...""</code> fenced block in the prompt.
Interview Question Suggester	<code>ApplicationsService.findById(dto.applicationId)</code> for <code>cvExtractedText</code> , then <code>JobsService.findById(application.jobPostingId)</code> for title/responsibilities/requiredSkills.	<code>cvSummary</code> falls back to the literal string <code>'No CV text available.'</code> via <code>??</code> if the application has no extracted text yet (so the prompt is never built with <code>undefined/null</code> interpolated into it). <code>interviewType</code> selects one of three hardcoded <code>typeGuidance</code> strings that get appended to the prompt.
Offer Letter Drafter	None from the DB — <code>GenerateOfferLetterDto</code> (candidateName, roleTitle, salary, startDate, probationPeriod, benefits, companyName) comes straight from the request body.	<code>benefits</code> array joined with <code>, `</code> . No entity is read or written by this endpoint — it is explicitly documented as <i>not</i> touching the offer entity's status; only the separate <code>OffersService.approveAndSend()</code> call (never invoked by AI code) transitions an offer to sent.

2.2 Patterns shared across all four features

- **No shared prompt-building helper / no truncation anywhere.** Each `buildXPrompt` function is a standalone template literal in `prompts.ts` ; there is no common "context assembler" abstraction, and none of the four prompts truncates or summarizes long input (e.g. a very long CV) before interpolating it — the full extracted text is always sent as-is.
- **No sanitization of user input before interpolation.** Field values (job titles, CV text, culture notes, benefits) are interpolated directly into the template string with no escaping, HTML-stripping, or prompt-injection filtering. The CV-scorer prompt instead defends against one specific risk at the *instruction* level — see §4.2 — rather than by sanitizing the input text itself. Response-side, `AIService.parseJson()` strips ````json` / ````` fences defensively before calling `JSON.parse` , since models frequently wrap structured output in

fences despite being told not to. Downstream Nest DTO validation (`class-validator` decorators on the response DTOs) is the actual enforcement point for output shape, not the prompt string itself.

- **Fixed numeric constraints, not truncation limits.** The "context management" discipline here is about output size, not input size: every prompt pins list lengths to exact numbers (5 `topSkills`, 3 `topStrengths`, 2 `topGaps`, 9 interview questions, 5–7 responsibilities) to keep responses within fixed Nest DTO shapes, as called out in the file-header comment in `prompts.ts` .
- **Single point of API contact.** `AIService` is documented as "the *only* place in the codebase that touches `@google/generative-ai` or reads `GEMINI_API_KEY` " — all four controllers' prompt strings flow through one `generateJson<T>()` method, so context handling, retries, and JSON parsing are centralized even though prompt construction is not.

3. Fallback / Degradation Behavior

`AIService.generateJson<T>()` never throws. It returns a discriminated union `AiResult<T> — { ok: true, data: T }` or `{ ok: false, reason, message }` — defined in `ai.types.ts` , with `AiFailureReason` values `'rate_limited' | 'timeout' | 'invalid_response' | 'error'` . Handling:

- **Timeout:** `withTimeout()` races the Gemini call against a 15,000 ms (`REQUEST_TIMEOUT_MS`) timer; if the timer wins, the promise rejects with `Error('timeout')` , which `classifyError()` maps to `reason: 'timeout'` .
- **Rate limiting:** `classifyError()` checks `err.status === 429` or a `/429|rate.?limit|quota/i` regex match on the error message, mapping to `reason: 'rate_limited'` .
- **Invalid JSON:** `parseJson()` strips leading/trailing `json /` fences, then calls `JSON.parse` ; on a parse failure it returns `reason: 'invalid_response'` .
- **Retry:** `isRetryable()` treats `'rate_limited'` , `'timeout'` , and `'error'` as retryable (but not `'invalid_response'` — a malformed response is treated as deterministic, not transient). `generateJson()` performs exactly **one** retry, after a fixed `RETRY_BACKOFF_MS` (1,200 ms) sleep, then returns whatever the second attempt yields.
- **No API key configured:** the constructor logs a warning and leaves `this.model = null` ; `generateJson()` short-circuits to `{ ok: false, reason: 'error', message: 'AI service is not configured' }` without attempting a network call.

3.1 Per-feature fallback (from `AiController`)

Job Description Generator — returns a clearly-labelled blank template instead of blocking job creation:

```
{
  roleSummary: `[AI unavailable – draft manually] ${dto.jobTitle} on the ${dto.department} team.`,
  keyResponsibilities: ['[Add key responsibility]'],
  requiredQualifications: dto.requiredSkills.map((s) => `Experience with ${s}`),
  preferredQualifications: ['[Add preferred qualification]'],
  whatWeOffer: ['[Add compensation/benefits/culture details]'],
  isFallback: true,
}
```

CV Screening & Scorer — the designated graceful-degradation feature. `aiScore` is left `null` (never defaulted to 0 or guessed) and the endpoint returns `scored: false` instead of a 500, so a CV scoring outage never removes a candidate from a recruiter's queue:

```
{
  scored: false,
  message: `AI scoring unavailable (${result.reason}) – this application requires manual
review.`,
}
```

Verified in `backend/test/ai.e2e-spec.ts` line 269: *"degrades gracefully (scored=false, aiScore stays null) when AI fails — designated fallback feature"*, asserting `res.body.message` matches `/manual review/i`.

Interview Question Suggester — falls back to a static, hardcoded question bank per interview type, defined in `interview-question-fallbacks.ts` (`INTERVIEW_QUESTION_FALLBACKS`), e.g. the `behavioural` set opens with *"Tell me about a time you disagreed with a teammate. How did you handle it?"* across 9 fixed questions per type. `ai.e2e-spec.ts` line 379–381 asserts `res.body.isFallback === true` and that the returned questions equal `INTERVIEW_QUESTION_FALLBACKS.behavioural` verbatim.

Offer Letter Drafter — returns a manually-completable draft assembled from the raw DTO fields, clearly marked unavailable:

```
{
  letterText:
    `[AI unavailable – draft manually]\n\n` +
    `Dear ${dto.candidateName},\n\n` +
    `We are pleased to offer you the position of ${dto.roleTitle} at ${dto.companyName}, ` +
    `starting ${dto.startDate}, with an annual salary of ${dto.salary} and a ` +
    `${dto.probationPeriod} probation period.\n\n` +
    `Benefits: ${dto.benefits.join(', ')} || '[list benefits]'\n\n` +
    `[Add conditions of employment and acceptance instructions]`,
  isFallback: true,
}
```

All four success paths set `isFallback: false`; `ai.e2e-spec.ts` asserts this flag on both success (lines 186, 362, 438) and failure (lines 204, 379, 455) branches.

4. Documented Prompt / Model Iterations

Two independent kinds of real iteration exist in this repository's history: (a) the AI provider/model migration, confirmed via `git log`, and (b) per-prompt wording iterations, documented verbatim in the repo's own `PROMPTS.md` (which explicitly states it exists so "a change to a prompt and its documented rationale never drift apart" with `prompts.ts`).

`git log --oneline --all -- backend/src/modules/ai PROMPTS.md` :

```
a8e31f3 changing to newer model
8f9c0c1 Set the RPM limit to 10
cbf2c48 shifted to Gemini
0c53947 Changed the AI model from Gemini to OpenRouter Gemini 4 flash free
b17ae7c add Gemini AI proxy, CV scoring, JD generation, interview questions, offer drafting,
and fallback handling
```

4.1 Iteration A — Provider/model migration (three real commits, confirmed by git diff)

b17ae7c → 0c53947 ("Changed the AI model from Gemini to OpenRouter Gemini 4 flash free"): initial implementation called `https://openrouter.ai/api/v1/chat/completions` with `OPENROUTER_MODEL = 'google/gemma-4-31b-it:free'`, using a hand-rolled `fetch()` call and manual `response.ok / status-429` checks.

0c53947 → cbf2c48 ("shifted to Gemini"): replaced the OpenRouter HTTP client with the `@google/generative-ai` SDK directly:

```
-const OPENROUTER_URL = 'https://openrouter.ai/api/v1/chat/completions';
-const OPENROUTER_MODEL = 'google/gemma-4-31b-it:free';
+const GEMINI_MODEL = 'gemini-2.0-flash';
```

This replaced the manual `fetch / response.json() / classifyHttpError()` path with `model.generateContent(prompt)` and `result.response.text()`, and renamed `openRouterApiKey` → `geminiApiKey` in `ConfigService`. *Rationale (inferred from commit message and surrounding context, not stated explicitly in the commit body):* moving off a free third-party proxy layer (OpenRouter) to Google's own SDK directly reduces an extra network hop and dependency, and gives direct access to Gemini's own rate-limit semantics instead of OpenRouter's re-exposed ones.

cbf2c48 → 8f9c0c1 ("Set the RPM limit to 10"):

```
-const GEMINI_MODEL = 'gemini-2.0-flash';
+const GEMINI_MODEL = 'gemini-2.5-flash-lite';
```

Rationale (stated in commit message): tuning to the free tier's 10 requests-per-minute limit — `gemini-2.5-flash-lite` was selected for the free-tier quota headroom it offers over `2.0-flash`.

8f9c0c1 → a8e31f3 ("changing to newer model"):

```
-const GEMINI_MODEL = 'gemini-2.5-flash-lite';
+const GEMINI_MODEL = 'gemini-2.5-flash';
```

alongside a small parsing/logging adjustment in the same commit — the retry-warning log line was changed to include `first.message` (the actual failure detail), not just `first.reason`:

```
-this.logger.warn(`Gemini call failed (${first.reason}), retrying once...`);
+this.logger.warn(
+  `Gemini call failed (${first.reason}): ${first.message} - retrying once...`,
+);
```

Rationale: the model constant name suggests moving to a stronger current model (`gemini-2.5-flash` over `-lite`), likely trading some free-tier throughput for better JSON-shape adherence and reasoning quality on the CV-scoring and interview-question features. The added `first.message` in the log is a genuine operational improvement — it makes rate-limit vs. timeout vs. auth failures distinguishable in logs without cross-referencing `reason` codes.

4.2 Iteration B — Prompt wording changes (documented in-repo, `PROMPTS.md`)

The repository's own `PROMPTS.md` documents pre-implementation "v1 attempt" wordings against the current ("v2 fix") prompts in `prompts.ts`, with stated rationale. Two representative examples:

CV Screening & Scorer — bias guardrail added:

```
v1 attempt:
"Here is a candidate's CV: ${cvText}
Here is the job: ${jobTitle}, skills: ${requiredSkills}
Rate this candidate and give feedback."
```

Problem (per `PROMPTS.md`): "Rate this candidate" with no scale produced scores as `"8/10"` in one run and `"72%"` in another, inconsistent with the `matchScore: 0-100` integer the DTO needs. More seriously, a test CV whose LinkedIn "about me" line mentioned an unrelated personal/community detail caused the model to comment on it in its "feedback" — a bias risk. Fix (current code): added as the prompt's first line, *"Judge only skills, experience, and demonstrated ability — never infer or comment on age, gender, ethnicity, nationality, or any protected characteristic, even if such details appear in the CV text,"* and pinned `matchScore` to an explicit `<integer 0-100>` scoped to *"how well their demonstrated skills/experience align... not a general CV quality score."*

Offer Letter Drafter — "draft, not sent" guardrail added:

```
v1 attempt:
"Write an offer letter for ${candidateName} for the ${roleTitle} role at
${companyName}, salary ${salary}, starting ${startDate}."
```

Problem (per `PROMPTS.md`): output sometimes read as an *already-sent, already-accepted* letter ("we're excited you've decided to join us"), dangerous given the feature's hard requirement that offers never auto-send; probation period and benefits were also silently dropped when not part of the model's idea of a "standard" letter. Fix (current code): added *"This is a DRAFT for a recruiter to review before sending — do not include any language implying the offer has already been sent or accepted,"* and listed every input field explicitly so none are dropped. The send-gate itself is also enforced in code (`OffersService.approveAndSend()` throws unless `status === 'draft'`), not just in the prompt.

Note on evidentiary status: the "v1 attempt" snippets and their described failure modes exist only as prose in `PROMPTS.md`, not as buildable prior commits of `prompts.ts` in git history (the git log for `prompts.ts` shows only the OpenRouter→Gemini header-comment rename, not a prompt-wording history). They are treated here as the project's own documented design rationale, not verified via `git diff` — flagged as **inference/secondary-source**, distinct from Iteration A which is independently confirmed via `git log -p`.

5. Data Privacy Implications

Every Gemini call in TalentBridge sends real candidate and job data to a third-party API with no redaction step in the code: the CV-scoring and interview-question prompts interpolate `application.cvExtractedText` — the full extracted CV text, including whatever name, contact details, education, employment history, or incidental personal information (e.g. community affiliations, addresses, dates of birth) the source document happens to contain — directly into the prompt string, and the offer-letter prompt sends the candidate's full name, salary, and start date. None of the four `buildXPrompt` functions strip, mask, or tokenize PII before sending it to Gemini; the only mitigation present in code is *instructional*, not technical — the CV-scoring prompt tells the model not to *comment on* protected characteristics it may see, but the underlying text (and thus the exposure to Google's API) still occurs regardless of whether the model complies. The primary risk surface is therefore: (1) whatever data-retention/training-use terms apply to the Gemini free tier the project uses (Google's standard terms for its free API tier permit using

submitted content to improve products, which is a materially different privacy posture than a paid enterprise tier with no-training guarantees — something this project has not configured or opted out of in code), and (2) transient exposure to a third party for every CV, job description, and offer letter processed, with no on-disk or in-transit anonymization. Mitigating factors that do exist: `AIService` is the single, centralized choke point for all outbound calls (so there is one place to add redaction if required), no data is persisted by Google-side (the app only stores what it gets back — `aiScore` , `topStrengths` , `topGaps` — not the prompt itself) in TalentBridge's own database, and the CV-scoring bias guardrail at least reduces (without eliminating) the chance that protected-characteristic data leaking into a CV surfaces in a recruiter-visible output field.